

Laporan Klasifikasi Teks dengan *Deep Learning* dan *Word Embedding*
IF5153 Pemrosesan Bahasa Alami
Praktikum 2



Oleh
Danendra Shafi Athallah
13523136

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung
Jl. Ganesha 10, Bandung 40132
2026/2027

Daftar Isi

1. Pendahuluan.....	3
2. Preprocessing dan Argumentasi.....	3
3. Word Embedding.....	5
4. Arsitektur Deep Learning.....	6
5. Perbandingan Hasil, Dampak, dan Strategi Inferensi.....	6
6. Insight.....	9

1. Pendahuluan

Praktikum 2 melanjutkan tugas klasifikasi sentimen tiga kelas pada dataset tweet politik India yang sama seperti Praktikum 1, dengan label 0 untuk Negatif, 1 untuk Netral, dan 2 untuk Positif. Pendekatannya kini deep learning yang dipadukan dengan word embedding. Data latih berjumlah 168.801 baris (dipecah stratified 90:10) dan data uji 72.344 baris.

Eksperimen dilakukan melalui tahap preprocessing dan tokenisasi, perbandingan beberapa word embedding, pelatihan CNN/RNN/LSTM, lalu inferensi untuk submission Kaggle. Prediksi akhir memakai ensemble CNN dengan bagging K-fold dan multi-seed, penyetelan pengali kelas pada out-of-fold, serta strategi inferensi sadar-duplikat yang dibahas pada Bagian 5. Seluruh keacakan dikunci pada seed 42 dan pelatihan embedding memakai satu worker agar hasil reproducible.

2. Preprocessing dan Argumentasi

Seluruh pembersihan dirangkum dalam fungsi `bersihkan_teks` pada Gambar 1. Dataset ini hasil scraping yang sudah huruf kecil dan minim tanda baca, tetapi masih menyimpan artefak encoding (mojibake).

```
In [8]: # TODO: process the dataset using the techniques from the previous Lab Work or other techniques (explain in the report).
```

```

berkembang menjadi kata negatif. misal: "Isn'de'kt'y"/"tsn'x80'x99t" -> "Isn't'y" sebelum tahap Lain,
- supaya negasi tidak hancur menjadi token yang tim "Isn'/\"didin\" (audit poin 5/6)
try:
    from ftfy import fix_text as _fix_text
    _PUNYA_FTFY = True
except Exception:
    _PUNYA_FTFY = False

def _perbaiki_encoding(s):
    s = str(s)
    if _PUNYA_FTFY:
        return _fix_text(s)
    # fallback bila ftfy tak tersedia: petakan urutan majemuk apostrof/kutip yang umum
    for a, b in [('"\"', '\"'), ('\"\"', '\"'), ('\"\"\", \"\"), ('\"\"\"\", \"\"), ('\"\"\"\"\", \"\"),
                ('\"\"\"\"\"\", \"\"), ('\"\"\"\"\"\", \"\"), ('\"\"\"\"\"\", \"\")]:
        s = s.replace(a, b)
    return s

# kontraksi tanpa apostrof (setelah apostrof dibuang cleaning); utamakan menjaga kata negasi
KONTRAKSI = {
    "dont": "do not", "dintd": "did not", "doesnt": "does not", "cant": "can not",
    "cannot": "can not", "wont": "will not", "wouldnt": "would not", "couldnt": "could not",
    "shouldnt": "should not", "isnt": "is not", "arent": "are not", "wasnt": "was not",
    "werent": "were not", "hasnt": "has not", "havent": "have not", "hadnt": "had not",
    "aint": "is not", "neednt": "need not", "mustnt": "must not", "mightnt": "might not",
    "wouldve": "would have", "couldve": "could have", "shouldve": "should have"
}

# emoji sentimen dipetakan ke token kata sebelum karakter non-alfabet dibuang
EMOJI = {
    "\U0001F602": "emojilucu", "\U0001F923": "emojilucu", "\U0001F60A": "emojisenang",
    "\U0001F601": "emojisenang", "\U0001F60D": "emojininta", "\U2764": "emojininta",
    "\U0001F4AD": "emojibagus", "\U0001F525": "emojibagus", "\U0001F621": "emojimarah",
    "\U0001F620": "emojimarah", "\U0001F622": "emojisedih", "\U0001F62D": "emojisedih",
    "\U0001F4AE": "emojijelek",
}

# cleaning: perbaiki encoding -> huruf kecil -> jaga emoji -> buang non-alfanumerik -> elangasi -> kontraksi
def bersihkan_teks(teks):
    teks = _perbaiki_encoding(teks).lower()
    for emo, tok in EMOJI.items():
        teks = teks.replace(emo, tok)
    teks = re.sub(r"(?![a-z0-9])+", "", teks)
    teks = re.sub(r"(?![\.\|\!|?], \")", "", teks)
    teks = ", ".join(KONTRAKSI.get(kata, kata) for kata in teks.split())
    teks = re.sub(r"(?![a-z]+)", "", teks).strip()
    return teks

# buat nilai kosong pada kolom teks (baris uji tidak boleh dibuang agar jumlah prediksi tepat)
train_df["Text"] = train_df["Text"].fillna("").astype(str)
test_df["Text"] = test_df["Text"].fillna("").astype(str)
train_df["clean_text"] = train_df["Text"].apply(bersihkan_teks)
test_df["clean_text"] = test_df["Text"].apply(bersihkan_teks)

# tokenizer hanya di fit pada data latih untuk menghindari kebocoran informasi dari data uji
tokenizer = Tokenizer(num_words=MAX_NUM_WORDS, oov_token='<OOV>')
tokenizer.fit_on_texts(train_df["clean_text"])
word_index = tokenizer.word_index
 vocab_SIZE = max(MAX_NUM_WORDS, len(word_index) + 1)

# ubah teks menjadi urutan index lalu memadamkan panjang (padding) di akhir kalimat
X_full = pad_sequences(tokenizer.texts_to_sequences(train_df["clean_text"]),
                        maxlen=MAX_LEN, padding="post", truncating="post")
X_test = pad_sequences(tokenizer.texts_to_sequences(test_df["clean_text"]),
                       maxlen=MAX_LEN, padding="post", truncating="post")
y_full = train_df["Sentiment"].values

# korpus token tanpa label untuk melatih Word2Vec & FastText: dengan train + test
# (label tidak dipakai, jadi bukan kebocoran) agar vektor juga mencakup kosakata data uji
korpus_teks = pd.concat([train_df["clean_text"], test_df["clean_text"]], ignore_index=True)
train_sentences = [teks.split() for teks in korpus_teks if teks.strip()]

X_tr, X_val, y_tr, y_val = train_test_split(
    X_full, y_full, test_size=0.1, random_state=RANDOM_STATE, stratify=y_full
)

print("ftfy tersedia:", _PUNYA_FTFY, "| Ukuran kosakata (vocab_size):", VOCAB_SIZE)
print("Jumlah kalimat korpus embedding (train + test):", len(train_sentences))
print("Bentuk data latih:", X_tr.shape, "| validasi:", X_val.shape, "| uji:", X_test.shape)
print("Distribusi label pada data latih:", np.bincount(y_tr))

```

Gambar 1. Fungsi preprocessing: perbaikan encoding, ekspansi kontraksi, pemetaan emoji, dan normalisasi elongasi.

Langkah pertama adalah memperbaiki encoding, karena sekitar 7,8% baris mengandung mojibake. Kalau pembuangan karakter non-alfanumerik dijalankan lebih dulu, bentuk rusak seperti “isn’t” berubah menjadi token yatim “isn” dan negasinya hilang pada sekitar 2% baris. Saya memakai ftfy bila tersedia, dengan pemetaan regex sebagai cadangan. Setelah itu teks disamakan menjadi huruf kecil, kontraksi seperti “dont” dan “isnt” dikembalikan menjadi “do not” dan “is not” supaya kata “not” tetap terbaca, emoji sentimen dipetakan ke token kata sebelum simbol lain dibuang, dan huruf berulang seperti “gooooo” diciutkan menjadi “good”.

Baris berteks kosong tidak dibuang, hanya diisi string kosong, supaya jumlah prediksi tetap 72.344 sesuai format submission. Tokenizer di-fit hanya pada data latih, dengan `num_words = 40.000` dan token “<OOV>”, agar tidak ada informasi data uji yang bocor ke pelatihan. Panjang urutan dipatok 50 token; distribusi panjang data uji tergolong pendek (median 11, persentil 99 = 39), dan pada `MAX_LEN = 50` hanya 0,46% baris yang terpotong, sehingga menaikkannya lagi hampir tidak berdampak. Word2Vec dan FastText dilatih pada gabungan teks train dan test tanpa memakai labelnya selama 12 epoch, sehingga vektornya turut mencakup kosakata yang hanya muncul di data uji.

3. Word Embedding

Saya membandingkan empat teknik embedding pada arsitektur CNN yang sama, lalu menambahkan gabungan GloVe dan FastText. Gambar 2 menampilkan sel perbandingannya dan Tabel 1 merangkum akurasi.

Word2Vec dan FastText dilatih langsung pada korpus dan keduanya mencakup seluruh kosakata, sedangkan GloVe memuat vektor Twitter-200 pra-latih dan SVD dibentuk dari matriks ko-okurensi lalu direduksi dengan Truncated SVD. Word2Vec menangkap relasi semantik dari konteks lokal, dan FastText menambahkan n-gram karakter sehingga lebih tahan pada kata OOV, salah ketik, serta slang. GloVe membawa statistik ko-okurensi global yang matang, tetapi kosakatanya tetap sehingga hanya sekitar 79,5% token dataset ini yang tercakup dan sisanya menjadi vektor nol. SVD berbeda paradigma karena representasinya count-based dan statis, dengan variansi menjelaskan 95,26%. Terakhir, GloVe dan FastText saya gabungkan dengan meng-concat kedua vektor menjadi 300 dimensi untuk memadukan statistik global dan ketahanan sub-kata, dan kombinasi inilah yang memberi hasil validasi terbaik.

```
In [13]: # TODO: Build and train a CNN Model

embedding_matrices = {
    "word2vec": embedding_matrix_word2vec,
    "GloVe": embedding_matrix_glove,
    "FastText": embedding_matrix_fasttext,
    "SVD": embedding_matrix_svd,
    # gabungan GloVe (statistik global) + FastText (sub-kata, tahan OOV) -> 200 dimensi
    "GloVe+FastText": np.concatenate([embedding_matrix_glove, embedding_matrix_fasttext], axis=1),
}
hasil_evaluasi = []

# embedding diinisialisasi dari matriks pra-latih lalu ikut dilatih (fine tuning);
# dimensi diambil dari bentuk matriks agar embedding gabungan 200-dim juga didukung
def lapisan_embedding(matriks_embedding):
    return layers.Embedding(
        input_dim=matriks_embedding.shape[0],
        output_dim=matriks_embedding.shape[1],
        weights=matriks_embedding,
        trainable=True,
    )

# CNN teks dengan beberapa ukuran kernel (3, 4, 5) yang digabungkan
def bangun_cnn(matriks_embedding):
    masukan = layers.Input(shape=(MAX_LEN,))
    x = lapisan_embedding(matriks_embedding)(masukan)
    x = layers.SpatialDropout1D(0.3)(x)
    cabang = [layers.GlobalMaxPooling1D()(layers.Conv1D(128, k, activation="relu")(x))
              for k in (3, 4, 5)]
    x = layers.Concatenate()(cabang)
    x = layers.Dropout(0.5)(x)
    x = layers.Dense(64, activation="relu")(x)
    keluaran = layers.Dense(NUM_CLASSES, activation="softmax")(x)
    model = models.Model(masukan, keluaran)
    model.compile(loss="sparse_categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
    return model

# pelatihan + evaluasi terpakai ulang untuk seluruh arsitektur agar perbandingan adil
def latih_dan_evaluasi(fungsi_bangun, matriks_embedding, nama_arsitektur, nama_embedding):
    tf.keras.utils.set_random_seed(RANDOM_STATE)
    model = fungsi_bangun(matriks_embedding)
    early_stop = EarlyStopping(monitor="val_accuracy", patience=3, restore_best_weights=True)
    model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=MAX_EPOCHS,
            batch_size=BATCH_SIZE, callbacks=[early_stop], verbose=0)
    prediksi_val = model.predict(X_val, verbose=0).argmax(axis=-1)
    akurasi = accuracy_score(y_val, prediksi_val)
    f1_makro = f1_score(y_val, prediksi_val, average="macro")
    hasil_evaluasi.append({"arsitektur": nama_arsitektur, "embedding": nama_embedding,
                          "val_accuracy": akurasi, "val_macro_f1": f1_makro})
    print("%-5s + %-14s -> val_accuracy=%4f | val_macro_f1=%4f" %
          (nama_arsitektur, nama_embedding, akurasi, f1_makro))
    return model

# bandingkan seluruh teknik embedding memakai arsitektur CNN yang sama
model_cnn_per_embedding = {}
for nama_embedding, matriks in embedding_matrices.items():
    model_cnn_per_embedding[nama_embedding] = latih_dan_evaluasi(
        bangun_cnn, matriks, "CNN", nama_embedding)

# embedding terbaik berdasarkan akurasi validasi pada CNN
hasil_cnn = [h for h in hasil_evaluasi if h["arsitektur"] == "CNN"]
embedding_terbaik = max(hasil_cnn, key=lambda h: h["val_accuracy"])["embedding"]
embedding_matrix_terbaik = embedding_matrices[embedding_terbaik]
model_cnn = model_cnn_per_embedding[embedding_terbaik]
print("\nEmbedding terbaik untuk CNN:", embedding_terbaik)

WARNING: All log messages before absl:InitializeLog() is called are written to STDERR
I0000 00:00:1789107490.990582    58 gpu_device.cc:2020] Created device /job:localhost/replica:0/task:0/device:GPU:0 with 13756 MB memory: -> device: 0, n
ame: Tesla T4, pci bus id: 0000:00:04.0, compute capability: 7.5
I0000 00:00:1789107490.993436    58 gpu_device.cc:2020] Created device /job:localhost/replica:0/task:0/device:GPU:1 with 13756 MB memory: -> device: 1, n
ame: Tesla T4, pci bus id: 0000:00:05.0, compute capability: 7.5
I0000 00:00:1789107498.676054    216 device_compiler.h:196] Compiled cluster using XLA! This line is logged at most once for the lifetime of the process.

CNN + Word2Vec -> val_accuracy=0.8591 | val_macro_f1=0.8515
CNN + GloVe -> val_accuracy=0.8622 | val_macro_f1=0.8551
CNN + FastText -> val_accuracy=0.8635 | val_macro_f1=0.8562
CNN + SVD -> val_accuracy=0.4274 | val_macro_f1=0.1996
CNN + GloVe+FastText -> val_accuracy=0.8661 | val_macro_f1=0.8592

Embedding terbaik untuk CNN: GloVe+FastText
```

Gambar 2. Sel perbandingan lima representasi embedding pada CNN; keluaran menunjukkan GloVe+FastText terbaik.

Embedding (arsitektur CNN)	Val. Accuracy	Val. Macro-F1
GloVe + FastText	0,8661	0,8592
FastText	0,8635	0,8562
GloVe (Twitter-200)	0,8622	0,8551
Word2Vec	0,8591	0,8515
SVD	0,4274	0,1996

Tabel 1. Performa validasi lima representasi embedding memakai arsitektur CNN yang sama.

4. Arsitektur Deep Learning

Ketiga arsitektur memakai lapisan Embedding yang diinisialisasi dari matriks pra-latih dan tetap dilatih (trainable), optimizer Adam, loss sparse categorical crossentropy, EarlyStopping, dan ReduceLROnPlateau. CNN memakai konvolusi banyak-kernel (3, 4, 5) dengan global max pooling, sedangkan RNN dan LSTM memakai lapisan dua arah (Gambar 3 menampilkan sel LSTM). Keempat embedding diuji lebih dahulu pada CNN, lalu embedding terbaik dipakai ulang untuk RNN dan LSTM agar pengaruh arsitektur terisolasi. Peringkat lengkapnya ada pada Tabel 2.

```
In [18]: # TODO: Build and train an LSTM model

# LSTM dua arah memakai embedding terbaik hasil perbandingan pada CNN
def bangun_lstm(matriks_embedding):
    masukan = layers.Input(shape=(MAX_LEN,))
    x = lapisan_embedding(matriks_embedding)(masukan)
    x = layers.SpatialDropout1D(0.3)(x)
    x = layers.Bidirectional(layers.LSTM(64))(x)
    x = layers.Dropout(0.5)(x)
    x = layers.Dense(64, activation="relu")(x)
    keluaran = layers.Dense(NUM_CLASSES, activation="softmax")(x)
    model = models.Model(masukan, keluaran)
    model.compile(loss="sparse_categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
    return model

model_lstm = latih_dan_evaluasi(bangun_lstm, embedding_matrix_terbaik, "LSTM", embedding_terbaik)

# seluruh eksperimen (kombinasi embedding x arsitektur) diurutkan dari akurasi tertinggi
tabel_hasil = pd.DataFrame(hasil_evaluasi).sort_values("val_accuracy", ascending=False).reset_index(drop=True)
print(tabel_hasil.to_string(index=False))
```

Gambar 3. Sel arsitektur LSTM dua arah (Bidirectional LSTM).

Median panjang teks hanya 11 token, sehingga sebagian besar tweet relatif pendek. Pada eksperimen ini CNN memberikan hasil terbaik, sedangkan LSTM dan RNN berada di bawahnya. Hasil tersebut mengindikasikan model dengan convolution sudah cukup efektif untuk dataset ini, meskipun bukan berarti dependensi urutan tidak berpengaruh sama sekali.

5. Perbandingan Hasil, Dampak, dan Strategi Inferensi

Tabel 2 merangkum keenam eksperimen (di luar SVD yang gagal), diurutkan dari akurasi validasi tertinggi.

Arsitektur	Embedding	Val. Accuracy	Val. Macro-F1
CNN	GloVe + FastText	0,8661	0,8592
CNN	FastText	0,8635	0,8562
CNN	GloVe (Twitter-200)	0,8622	0,8551
LSTM	GloVe + FastText	0,8593	0,8516
CNN	Word2Vec	0,8591	0,8515

Arsitektur	Embedding	Val. Accuracy	Val. Macro-F1
RNN	GloVe + FastText	0,8346	0,8251

Tabel 2. Hasil validasi seluruh kombinasi embedding dan arsitektur (baris terbaik ditebalkan).

Perbedaan ketiga embedding padat kecil, sekitar 0,004 pada accuracy, dan gabungan GloVe+FastText berada tipis di atasnya (0,8661). SVD jauh tertinggal (0,4274; macro-F1 0,1996), kemungkinan karena representasinya yang berbeda dari embedding prediktif lain. Antar-arsitektur selisihnya sedikit lebih besar: CNN (0,8661) di atas LSTM (0,8593) dan RNN (0,8346), sehingga pada eksperimen ini pilihan arsitektur terasa lebih berpengaruh daripada pilihan embedding padat.

Prediksi akhir memakai ensemble CNN dari bagging StratifiedKFold dan beberapa seed, dengan pengali per-kelas yang disetel pada prediksi out-of-fold untuk mengimbangi kelas yang timpang; estimasi akurasi out-of-fold model akhir 0,8678. Selain penyetelan model, saya memanfaatkan kemiripan antara data latih dan uji yang dirangkum pada Tabel 3.

Sekitar 14,92% baris uji identik persis dengan baris latih dan 15,56% identik setelah normalisasi, dengan konflik label yang kecil (78 key). Untuk baris uji yang teksnya sudah ada di data latih, saya memakai label dari data latih sebagai prediksi, dengan pencocokan exact lebih dahulu lalu normalized, mengambil suara terbanyak dan mengabaikan kasus seri. Sel inferensinya ditampilkan pada Gambar 4.

```
In [ ]: # TODO: Generate predictions on test.csv and save to submission format (you can use the example code below if you want)

# Strategi inferensi:
# - embedding gabungan GloVe-300 + FastText-100 (300-dim), kombinasi terbaik pada validasi;
# - ensemble CNN dgn bagging StratifiedFold + multi-seed (RNN/LSTM/GRU tidak menambah akurasi);
# - ReduceLROnPlateau + latih lebih lama; tuning pengali per kelas pada OOF;
# - DUPLICATE-AWARE OVRSHIDE: teks uji identik dgn train memakai label train (faktor terbesar).
embedding_matrix_final = np.concatenate(
    [embedding_matrix_glove, embedding_matrix_fasttext], axis=1)
arsitektur_ensemble = [{"CNN", bahasa_usa}]

# latih tiap fold x seed; rata-rata probabilitas pada X target; kumpulkan OOF untuk tuning kelas
def latih_ensemble(X_data, y_data, X_target, seeds, arsitektur, n_splits=N_SPLITS,
                  kembalikan_oof=False):
    kembalikan_oof = False
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=RANDOM_STATE)
    kumpulan_prob = []
    oof = np.zeros((len(X_data), NUM_CLASSES), dtype="float32")
    oof_hitung = np.zeros(len(X_data), dtype="float32")
    for fold, (idx_tr, idx_va) in enumerate(skf.split(X_data, y_data), start=1):
        for nama, fungsi in arsitektur:
            for seed in seeds:
                tf.keras.utils.set_random_seed(seed)
                model = fungsi(embedding_matrix_final)
                callbacks = [
                    EarlyStopping(monitor="val_accuracy", patience=4, restore_best_weights=True),
                    ReduceLROnPlateau(monitor="val_accuracy", factor=0.5, patience=2, min_lr=1e-5),
                ]
                model.fit(X_data[idx_tr], y_data[idx_tr],
                        validation_data=(X_data[idx_va], y_data[idx_va]),
                        epochs=MAX_EPOCHS, batch_size=BATCH_SIZE,
                        callbacks=callbacks, verbose=0)
                kumpulan_prob.append(model.predict(X_target, verbose=0))
            if kembalikan_oof:
                oof[idx_va] += model.predict(X_data[idx_va], verbose=0)
                oof_hitung[idx_va] += 1
            print(" fold %d/%d selesai" % (fold, n_splits))
    prob_target = np.mean(kumpulan_prob, axis=0)
    if kembalikan_oof:
        return prob_target, oof / np.maximum(oof_hitung[1, None], 1.0)
    return prob_target

# pengali per-kelas (coordinate ascent) yang memaksimalkan akurasi pada OOF
def cari_pengali(oof, y, putaran=3):
    pengali = np.ones(NUM_CLASSES, dtype="float32")
    grid = np.linspace(0.5, 1.4, 17)
    akurasi = accuracy_score(y, oof.argmax(axis=1))
    for c in range(putaran):
        for c in range(NUM_CLASSES):
            for g in grid:
                kand = pengali.copy(); kand[c] = g
                a = accuracy_score(y, (oof * kand).argmax(axis=1))
                if a > akurasi:
                    akurasi, pengali = a, kand
    return pengali, akurasi

# ensemble final: CNN, bagging K-fold penuh + multi-seed, langsung pada seluruh data latih
print("Ensemble final (CNN, K-fold, multi-seed)...")
prob_final, oof_final = latih_ensemble(X_full, y_full, X_test, ENSEMBLE_SEEDS,
                                      arsitektur_ensemble, kembalikan_oof=True)

# tuning pengali per-kelas pada OOF untuk membagi ketimpangan kelas
pengali, akurasi_oof = cari_pengali(oof_final, y_full)
print("Akurasi OOF: default %.4f -> setelah tuning kelas %.4f | pengali-%s" %
      (accuracy_score(y_full, oof_final.argmax(axis=1)), akurasi_oof, np.round(pengali, 3)))
predicted_classes = (prob_final * pengali).argmax(axis=1)

# ---- DUPLICATE-AWARE OVRSHIDE (sudut poin 1-3: Faktor terbesar) ----
# teks uji yang identik dgn train memakai label train (exact dulu, lalu normalisasi; meri dilewati)
if OOF_COMPARE:
    from collections import Counter as _Counter, defaultdict as _dd
    def _norm(t):
        t = re.sub(r"[a-z0-9@]", "", str(t).lower())
        return re.sub(r"[^a-z]", "", t, strip=True)
    peta_exact, peta_norm = _dd(list), _dd(list)
    for i, y in zip(train_df["text"], y_full):
        peta_exact[str(i)].append(int(y))
        k = _norm(t)
        if k:
            peta_norm[k].append(int(y))
    def _vote(lst):
        c = _Counter(lst).most_common()
        return None if (len(c) > 1 and c[0][1] == c[1][1]) else c[0][0]
    pred_final = defaultdict_classes.copy(); n_exact = n_norm = 0
    for i, t in enumerate(test_df["text"]):
        ts = str(t)
        if ts in peta_exact:
            v = _vote(peta_exact[ts])
            if v is not None:
                if pred_final[i] != v: n_exact += 1
                pred_final[i] = v; continue
        k = _norm(t)
        if k in peta_norm:
            v = _vote(peta_norm[k])
            if v is not None:
                if pred_final[i] != v: n_norm += 1
                pred_final[i] = v
    print("Override duplikat -> exact ubah %d, normalized tambah %d, total %d (%.2f% test)" %
          (n_exact, n_norm, n_exact + n_norm, 100 * (n_exact + n_norm) / len(test_df)))
    predicted_classes = pred_final

# format to match sample submission.csv
submission_df = pd.DataFrame(
    {"id": test_df["id"],
     "sentiment": predicted_classes.astype(int),
    })
submission_df.to_csv("submission.csv", index=False)
print("submission.csv tersimpan dengan", len(submission_df), "baris.")
```

Gambar 4. Sel inferensi final: ensemble CNN K-fold multi-seed, penyeteralan pengali kelas, dan override sadar-duplikat.

Metrik overlap train–test (terukur)	Nilai
Baris uji identik persis dengan train	10.795 (14,92%)
Baris uji identik setelah normalisasi	11.255 (15,56%)
Kesesuaian prediksi model final vs label train pada subset itu	±95,0%
Prediksi berubah bila override diterapkan (model final)	565 (0,78% test)
Duplikat internal train (indikasi OOF optimistik)	27.080 (16,04%)

Tabel 3. Analisis overlap train–test yang diukur langsung dari train.csv dan test.csv.

Model final sudah benar pada sekitar 95% baris duplikat, jadi override hanya mengubah 565 prediksi (0,78% data uji). Angka ini lebih kecil daripada versi model sebelumnya karena pipeline yang diperbaiki sudah menebak lebih banyak duplikat dengan benar. Cakupannya terukur, sedangkan besar kenaikan pada data uji tersembunyi masih berupa perkiraan.

6. Insight

- **Preprocessing.** Urutan perbaikan encoding sebelum tokenisasi penting karena beberapa negasi hilang ketika teks langsung dibersihkan.
- **Embedding.** Word2Vec, GloVe, dan FastText menghasilkan performa yang relatif dekat; kombinasi GloVe dan FastText memberikan hasil validasi terbaik, sedangkan SVD jauh di bawahnya.
- **Arsitektur.** Pada dataset ini CNN memperoleh validation accuracy tertinggi dibandingkan LSTM dan RNN.
- **Duplikasi data.** Ditemukan cukup banyak teks yang sama antara train dan test (sekitar 15,56%), sehingga informasi dari data latih dapat dimanfaatkan untuk sebagian sampel uji.
- **Validasi.** Duplikat di dalam train membuat skor cross-validation perlu diinterpretasikan dengan hati-hati karena teks identik dapat muncul pada fold pelatihan dan validasi.